

Software Design Patterns

If you wanna go fast, go alone. If you wanna go far...

Coding Patterns

Software engineering patterns are crucial for data engineers as they provide proven, reusable solutions to common challenges, ensuring consistency, maintainability, and scalability in data systems. By leveraging patterns, data engineers can design robust pipelines, optimize data processing, and enhance system reliability, while also making code easier to understand and modify.

These patterns facilitate collaboration among team members, reduce the risk of errors, and accelerate development by offering a structured approach to problem-solving, ultimately leading to more efficient and resilient data architectures.

The following slides heavily benefited from this [blog post](#).

1

Functional Design

Functional design centers on creating stateless, pure functions that produce predictable outputs without side effects, enhancing code clarity, maintainability, and scalability.

2

Factory Pattern

The Factory pattern simplifies object creation by abstracting the instantiation process, allowing flexible and dynamic object generation without specifying exact classes.

3

Strategy Pattern

The Strategy pattern enables runtime selection of algorithms by encapsulating them in interchangeable classes, promoting modular, maintainable, and extensible code.

4

Singleton Pattern

The Singleton pattern restricts a class to a single instance and provides a global access point, ensuring consistent resource management and centralized control across the system.

5

Conventions, styles, code hygiene & smells

Beyond patterns from software engineering, there are a bunch of conventions, styles and “unwritten rules” that you should adhere as a data engineer.

In functional programming, three key concepts—atomicity, idempotency, and no side effects—play a vital role in ensuring code reliability, predictability, and composability.

Atomicity

- › Definition: An operation is considered **atomic** if it is indivisible, meaning it either fully completes or does nothing at all, leaving the system in a consistent state.
- › In Functional Programming:
 - › Functions in functional programming aim to behave like atomic operations in that they take inputs, produce outputs, and do not leave intermediate or partial states.
 - › Example: A function like $f(x) = x + 1$ is atomic because given an input, it always produces a single, complete output without any transitional or partial results visible to the outside world.

Idempotency

- › Definition: A function is **idempotent** if calling it multiple times with the same input produces the same result every time.
- › In Functional Programming:
 - › Idempotency: ensures predictability and helps in reasoning about code.
 - › A function is idempotent if $f(f(x)) = f(x)$, e.g., $abs(x)$ (which returns the absolute value of x) is idempotent because: $abs(abs(-5))$ will always result in 5, regardless of how many times it is called.
 - › Idempotency is particularly useful in distributed systems and retry logic to prevent unintended effects.

No side effects

- › Definition: A function is said to have no side effects if it does not modify any state or interact with the outside world (e.g., no database updates, file writes, or global variable modifications).
- › In Functional Programming:
 - › Functions should be pure, meaning: They depend only on their input arguments.
 - › They produce no observable side effects.
 - › By contrast, a function that modifies a global variable or changes state of external dependencies violates this

In data engineering when we talk about idempotent function, we mean that running the same function on the same input data, only modifies the target (table) once

Staging table and function code

- › Initially we have a table with one row and want to **insert / update** (“upsert”) the data from the “staging table”:

Id	Name	Country
-----	-----	-----
02	Alex	France

Using this query, we end up with a non-idempotent data pipeline:

```
INSERT INTO main_table (id, name, country)
SELECT id, name, country
FROM stg_table;
```

Table state `main_table` across function applies

- › Initial State

Id	Name	Country
01	Jannic	Germany

- › First Function Apply “ $f(x)$ ”

Id	Name	Country
01	Jannic	Germany
02	Alex	France

- › Second Function Apply “ $f(f(x))$ ”

Id	Name	Country
01	Jannic	Germany
02	Alex	France
02	Alex	France

In data engineering when we talk about idempotent function, we mean that running the same function on the same input data, only modifies the target (table) once

Item potent transformation

- › Replacing this function with an idempotent one means we can apply it several times, but the result does not change:

```
INSERT OR REPLACE INTO  
main_table (id, name, value)  
SELECT id, name, value  
FROM stg_table;
```

If the staging table remains constant:

Id	Name	Country
02	Alex	France

Table state `main_table` across function applies

- › Initial State

Id	Name	Country
01	Jannic	Germany

- › First Function apply

Id	Name	Country
01	Jannic	Germany
02	Alex	France

- › Second Function Apply

Id	Name	Country
01	Jannic	Germany
02	Alex	France

In functional programming we prefer stateless functions over stateful ones

Stateless

- › A stateless system does not retain any information (state) between calls or interactions. Each request or function call is independent and does not depend on previous interactions.

```
def add_numbers(a, b):  
    return a + b
```

```
# Stateless: The function does not depend on or modify any external  
state.
```

```
print(add_numbers(3, 5)) # Output: 8  
print(add_numbers(10, 20)) # Output: 30
```

Stateful

- › A stateful system retains information (state) between calls or interactions. Each request or function call can depend on and modify the stored state.

```
class Counter:  
    def __init__(self):  
        self.count = 0 # Initial state  
  
    def increment(self):  
        self.count += 1  
        return self.count
```

```
# Stateful: The class keeps track of `count` across calls.  
counter = Counter()  
print(counter.increment()) # Output: 1  
print(counter.increment()) # Output: 2
```

Code Demo

Find the code demo here: https://gitlab.com/dsti-data-pipeline-2/design-patterns/-/blob/main/patterns/sample_etl_project/initial_design.py?ref_type=heads

A function which if executed multiple times keeps changing the result it violates the idempotency principle in data engineering, i.e., $f(f(x)) \neq f(x)$

Example Load Function

```
def load(data):
    conn = sqlite3.connect('./data/social_etl.db')
    cur = conn.cursor()
    try:
        for post in data:
            cur.execute(
                """INSERT INTO social_posts (
                    id, source, social_data
                ) VALUES (
                    :id, :source, :social_data
                )""", {
                    'id': post.get('id'),
                    'score': post.get('score'),
                    'social_data': str(
                        {
                            'title': post.get('title'),
                            'url': post.get('url'),
                            'comments': post.get('num_comments'),
                            'created': post.get('created'),
                            'text': post.get('selftext'),
                        }
                    )
                })
    finally:
        conn.commit()
        conn.close()
```

Is this function designed well?

- › **Atomicity**: No, it does two things:
 - › Managers database connections
 - › And loads data into database
- › **Idempotency**: No, if the function is applied multiple times, the data will be applied multiple times
- › **No side effects**: Load function has no side effects.

In the refactored version, the `db_conn` object is injected and the SQL query is redesigned to be idempotent by using **INSERT OR REPLACE**

Refactored load function

```
def load(social_data, db_conn) -> None:
    logging.info('Loading twitter data.')
    if db_conn is None:
        raise ValueError('db_cursor is None')

    cur = db_conn.cursor()
    try:
        for post in social_data:
            cur.execute("""
                INSERT OR REPLACE INTO social_posts (
                    id, source, social_data
                ) VALUES (
                    :id, :source, :social_data
                )
            """,
            {
                'id': post.id,
                'source': post.source,
                'social_data': str(asdict(post.social_data)),
            },
            )
    finally:
        cur.close()
        db_conn.commit()
```

Is this function designed well?

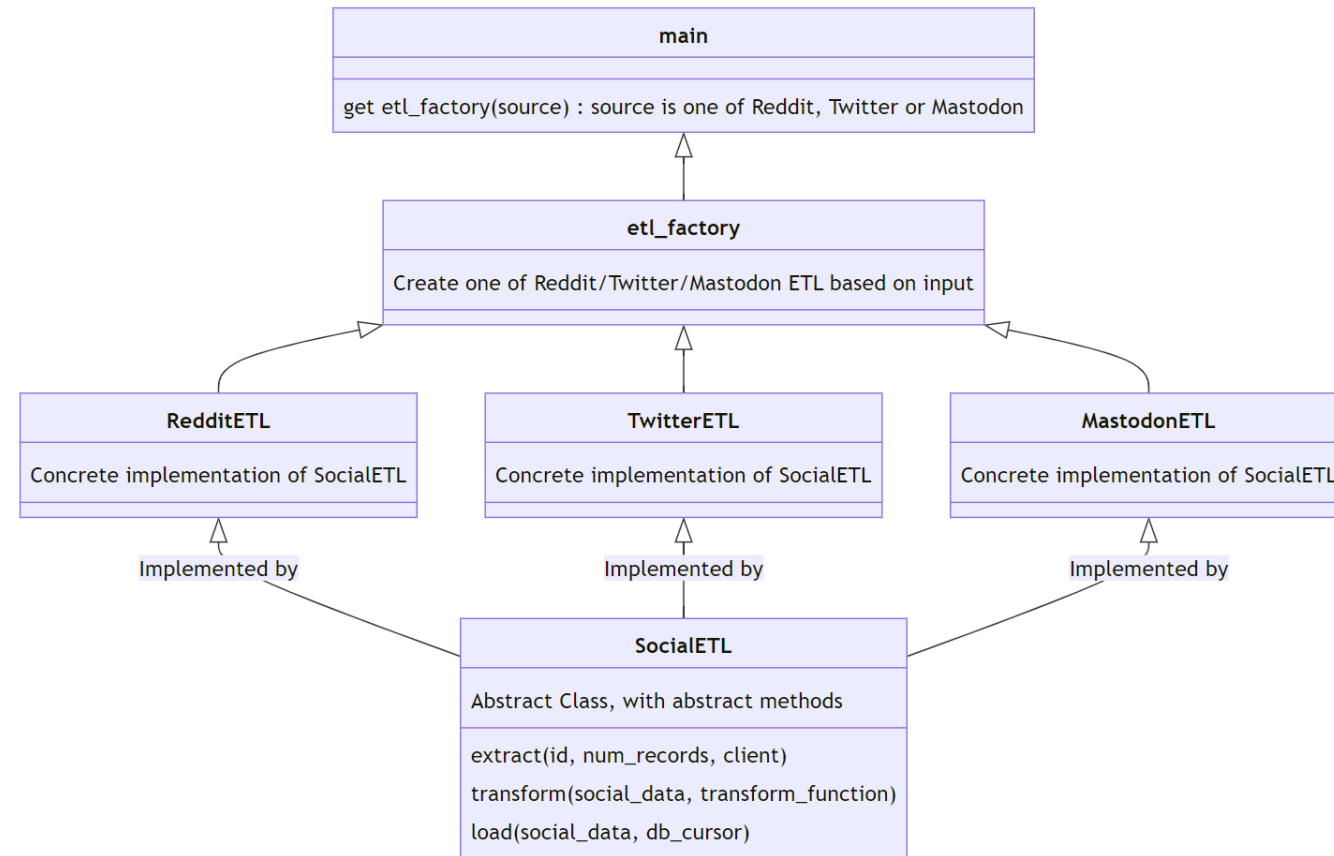
- › **Atomicity**: pretty atomic 😊 But the cursor could be created outside of the function using a context manager
- › **Idempotency**: Yes, if executed multiple times with the same input data (`social_data`) it will only modify the target table ones
- › **No side effects**: Yes. Note if you passed in the `cursor` and closed it, this would violate the no side effects principle

The Factory Design Pattern provides a method for creating objects without specifying their exact class, promoting loose coupling and flexibility in object creation.

Setup

- › Use a factory pattern when multiple pipelines follow a similar pattern.
- › A factory will be responsible for creating the appropriate (Reddit, Twitter, or mastodon) etl object. The code that calls the factory will use that elt object, unaware of its internal implementation.
- › Prevents complex if..else statements that are hard to manage and provides a standard interface to interact with multiple similar pipelines.
- › You can define a standard set of methods that all the ET classes must implement. The common method's names & signatures (inputs, & outputs) are called abstract interface

UML Factory Design Pattern Example



In a first step, the abstract interface of all social media ETL is defined. Next, concrete child classes implement all methods

Abstract Interface for social ETL

```
import os
from abc import ABC, abstractmethod # python module to define
abstract interfaces

class SocialETL(ABC):
    @abstractmethod
    def extract(self, id, num_records, client):
        pass

    @abstractmethod
    def transform(self, social_data):
        pass

    @abstractmethod
    def load(self, social_data, db_conn):
        pass

    @abstractmethod
    def run(self, db_conn, client, id, num_records):
        pass
```

Concrete implementation classes skeletons

```
# Concrete implementation of the abstract Class
class RedditETL(SocialETL):
    def extract(self, id, num_records, client):
        # code to extract reddit data

    def transform(self, social_data):
        # code to transform reddit data

    def load(self, social_data, db_conn):
        # code to load reddit data into the final table

    def run(self, db_conn, client, id, num_records):
        # code to run extract, transform and load

# Concrete implementation of the abstract Class
class TwitterETL(SocialETL):
    def extract(self, id, num_records, client):
        # code to extract reddit data

    def transform(self, social_data):
        # code to transform reddit data

    ...
```

Next, a factory creates the corresponding class instance to be used in the ETL process. The ETL pipeline using the instance needs not know anything about its inner workings.

ETL factory function

```
def etl_factory(source):
    factory = {
        'Reddit': (
            praw.Reddit(
                client_id=os.environ['REDDIT_CLIENT_ID'],
                client_secret=os.environ['REDDIT_CLIENT_SECRET'],
                user_agent=os.environ['REDDIT_USER_AGENT'],
            ),
            RedditETL(),
        ),
        'Twitter': (
            tweepy.Client(bearer_token=os.environ['BEARER_TOKEN']),
            TwitterETL(),
        ),
    }
    return factory[source]

# calling code
client, social_etl = etl_factory(source)
social_etl.run(db_conn, client, ...)
```

Using a factory decouples the implementation

- › Each implementing class has a `.run(...)` method. This allows to decouple the inner workings of each social media ETL from executing it.
- › This allows for consistency in your code base and makes it much easier to evolve pipelines
- › But don't follow this principle at all costs: if the pipelines differ too much, they should be separated; otherwise you risk “forcing” a pattern where it does not work (well).
- › Typical candidate for “premature optimization”: If you only have 1-3 types, do you really need this pattern?

The Strategy Design Pattern defines a family of algorithms, encapsulates them in separate classes, and allows them to be interchangeable within a context.

Set of transformation strategies

```
import random

def no_transformation(social_data):
    return social_data

def random_choice_filter(social_data):
    return random.choices(social_data, k=2)

def standard_deviation_outlier_filter(social_data):
    # code to only keep standard deviation based outlier
    return filtered_data

# a factory to return the appropriate transformation function
def transformation_factory(value):
    factory = {
        'sd': standard_deviation_outlier_filter,
        'no_tx': no_transformation,
        'rand': random_choice_filter,
    }
    return factory[value]
```

Transformation function is injected into the method

```
class Reddit(SocialETL):

    def transform(self, social_data, transform_function):
        """ Function to transform reddit data, by only keeping the
        posts with number of comments greater than 2 standard
        deviations away from the mean number of comments.
        """
        logging.info('Transforming reddit data.')
        return transform_function(social_data)

...
# Calling code
transformation = 'sd' # 'no_tx' or 'rand'
source = ...
client, social_etl = etl_factory(source)
db = ...
social_etl.run(
    db_cursor_context=db.managed_cursor(),
    client=client,
    transform_function=transformation_factory(transformation),
)
```

Beyond software engineering patterns, there are a host of coding practices, styles, conventions that you should follow (and code smells to avoid) in data engineering.

The most important ones will be discussed in this section. Apply them in your work. They let you stand out compared to data scientist / analysts

- ▶ **Context Managers:** Context managers are programming constructs that manage the setup and teardown of resources, ensuring they are properly acquired and released. In Python, they are implemented using the with statement, which provides a clean and concise way to work with resources like files or locks.
- ▶ **Dataclasses:** Dataclasses are a feature in Python that provide a decorator and functions for automatically adding special methods to user-defined classes. They simplify class creation by generating boilerplate code for methods like `__init__`, `__repr__`, and `__eq__` based on class attributes.
- ▶ **Magic Strings:** Magic strings are hard-coded string literals used directly in code, which can lead to errors and make maintenance difficult due to typos or inconsistencies. To avoid them, it's best to define constants or use enumerations, enhancing code readability and reducing the risk of mistakes.
- ▶ **Primitive Obsession:** This is a code smell that occurs when primitive data types (like strings, integers, lists, and dictionaries) are overused to represent complex concepts or entities in your code. Instead of using these native data structures in complex or nested forms, it's better to create custom classes or types that explicitly define and encapsulate the data and behavior.

Context managers in Python are used to manage resources efficiently, ensuring proper acquisition and release of resources, such as files, database connections, or locks.

Context manager example

```
import sqlite3

class DatabaseConnection:
    def __init__(self, db_name):
        self.db_name = db_name
        self.connection = None

    def __enter__(self):
        # Establish the database connection
        self.connection = sqlite3.connect(self.db_name)
        return self.connection

    def __exit__(self, exc_type, exc_value, traceback):
        # Commit changes and close the connection
        if self.connection:
            if exc_type is None: # No exceptions occurred
                self.connection.commit()
            else: # An exception occurred, rollback changes
                self.connection.rollback()
            self.connection.close()
```

Using a factory decouples the implementation

- › Context managers in Python are used to manage resources efficiently, ensuring proper acquisition and release of resources, such as files, database connections, or locks.
- › The primary way to use a context manager is via the with statement. This ensures that resources are properly cleaned up after use, even in the event of an error.
- › The key methods for implementing context managers are:
 - › `__enter__()`: Sets up the resource and returns it.
 - › `__exit__()`: Cleans up the resource, handling exceptions if necessary.

The primary way to use a context manager is via the `with` statement. This ensures that resources are properly cleaned up after use, even in the event of an error.

Context manager example

```
import pandas as pd

db_name = "example.db"

with DatabaseConnection(db_name) as conn:
    cursor = conn.cursor()

    # Create a table
    cursor.execute("""
    CREATE TABLE IF NOT EXISTS users
    (id INTEGER PRIMARY KEY, name TEXT)""")

    # Insert data
    cursor.execute("INSERT INTO users (name) VALUES (?)", ("Alice",))

    # Query data
    cursor.execute("SELECT * FROM users")
    df = pd.DataFrame(cursor.fetchall())
```

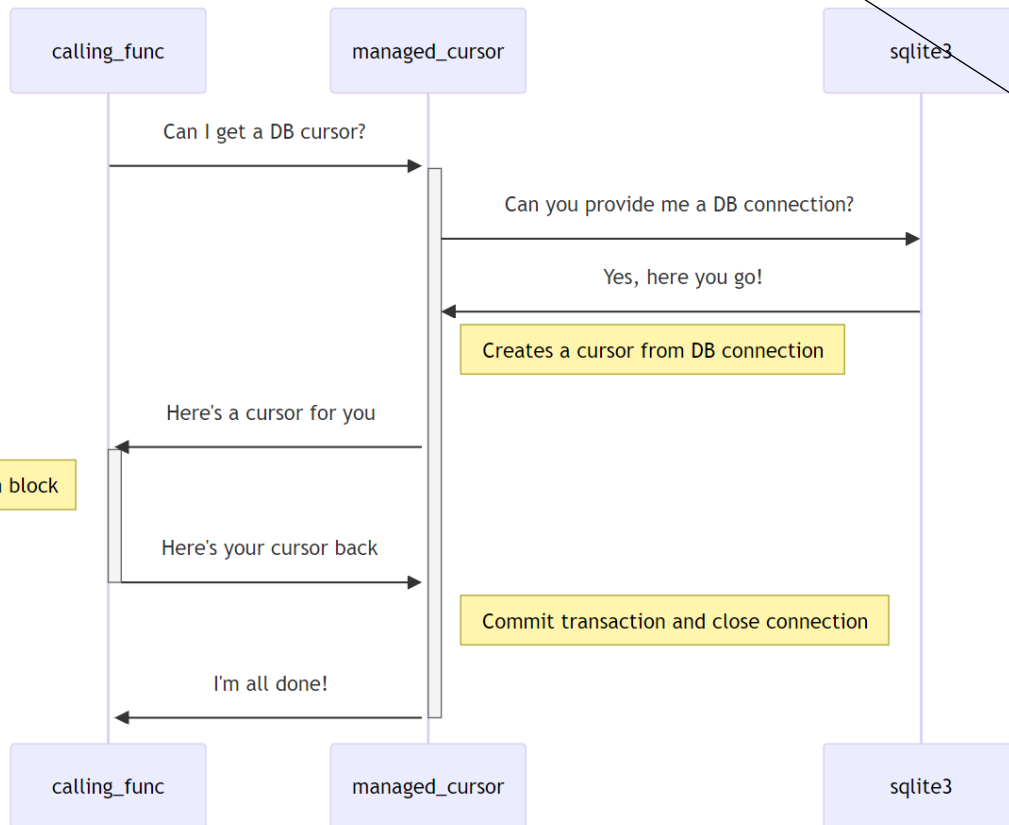
Key Benefits

- › **Encapsulation**: Resource management logic is encapsulated within the `DatabaseConnection` class.
- › **Error Handling**: Ensures proper cleanup with `__exit__()`, even in case of exceptions.
- › **Convenience**: Simplifies resource management with the `with` statement.

This makes context managers an excellent tool for writing clean, reliable, and robust Python code.

Alternatively, you can use Python's contextmanager decorator

Setup



```
from contextlib import contextmanager
```

```
class DatabaseConnection:
    def __init__(
        self, db_type: str = 'sqlite3', db_file: str = 'data/socialnet1.db'
    ) -> None:
        """Class to connect to a database.
        Args:
            db_type (str, optional): Database type. Defaults to 'sqlite3'.
            db_file (str, optional): Database file. Defaults to 'data/socialnet1.db'.
        """
        self._db_type = db_type
        self._db_file = db_file

    @contextmanager
    def managed_cursor(self) -> Iterator[sqlite3.Cursor]:
        """Function to create a managed database cursor.
        Yields:
            sqlite3.Cursor: A sqlite3 cursor.
        """
        if self._db_type == 'sqlite3':
            _conn = sqlite3.connect(self._db_file)
            cur = _conn.cursor()
            try:
                yield cur
            finally:
                _conn.commit()
                cur.close()
                _conn.close()

    def __str__(self) -> str:
        return f'{self._db_type}://{self._db_file}'
```

```
db = DatabaseConnection()
```

```
with db.managed_cursor() as cur: # cursor and connection are open
    cur.execute("YOUR SQL QUERY")
```

Code Demo

Find the code demo here: https://gitlab.com/dsti-data-pipeline-2/design-patterns/-/blob/main/patterns/more_patterns/data_classes.py?ref_type=heads

Native data types (`list`, `dict`, `set`) should never, ever, ever be nested! While they are intuitive to populate, it is difficult to maintain code operating on them

Config example using native data types, easy to setup...

```
config = {
    "app_name": "MyApp",
    "version": "1.0.0",
    "services": [
        {
            "name": "auth",
            "enabled": True,
            "endpoints": ["/login", "/register", "/logout"]
        },
        {
            "name": "data",
            "enabled": False,
            "endpoints": ["/fetch", "/upload"]
        }
    ],
    "users": [
        {
            "username": "admin",
            "roles": ["admin", "editor"]
        },
        {
            "username": "guest",
            "roles": ["viewer"]
        }
    ]
}
```

What if keys are named/deleted/added?

... But cumbersome to work with

```
def process_config(config):
    # Access application name and version
    app_name = config.get("app_name", "Unknown")
    version = config.get("version", "0.0.0")
    print(f"Processing config for {app_name} version {version}...\n")

    # Process services
    services = config.get("services", [])
    for service in services:
        service_name = service.get("name", "Unknown Service")
        is_enabled = service.get("enabled", False)
        print(f"Service: {service_name}, Enabled: {is_enabled}")

    # Access endpoints
    endpoints = service.get("endpoints", [])
    for endpoint in endpoints:
        print(f"Endpoint: {endpoint}")
```

You have to know all the keys. Incorrect key results in run time errors (=too late)

Instead, use classes for complex objects. Use **dataclasses** for objects that (mainly) “carry” data around in your code. Their typing adds safety to your code and makes it maintainable.

Define classes for each type of (complex) data

```
from dataclasses import dataclass, field

@dataclass
class Endpoint:
    path: str

@dataclass
class Service:
    name: str
    enabled: bool
    endpoints: list[str]

@dataclass
class User:
    username: str
    roles: list[str]

@dataclass
class AppConfig:
    app_name: str
    version: str
    services: list[Service] = field(default_factory=list)
    users: list[User] = field(default_factory=list)
```

Initialize the class which is now strongly typed

```
config_dataclass = AppConfig(
    app_name="MyApp",
    version="1.0.0",
    services=[
        Service(
            name="auth",
            enabled=True,
            endpoints=["/login", "/register", "/logout"]),
        Service(
            name="data",
            enabled=False,
            endpoints=["/fetch", "/upload"]),
    ],
    users=[
        User(username="admin", roles=["admin", "editor"]),
        User(username="guest", roles=["viewer"]),
    ]
)
```

Store the data in a json file and use an alternative constructor to create the dataclass instance carrying the data. Use this pattern instead of nested/mixed native datatypes.

Alternative constructor to read the json files

```
@dataclass
class AppConfig:
    app_name: str
    version: str
    services: list[Service] = field(default_factory=list)
    users: list[User] = field(default_factory=list)

    @classmethod
    def from_json(cls, json_data: dict):
        # Create list of Service objects
        services = [Service(**service)
                     for service in json_data.get("services", [])]

        # Create list of User objects
        users = [User(**user) for user in json_data.get("users", [])]

        # Return the AppConfig instance
        return cls(
            app_name=json_data["app_name"],
            version=json_data["version"],
            services=services,
            users=users
        )
```

Never nest/mix native data types! Ever! For real!

```
# only one type in a list
numbers: list[int] = [1, 2, 3, 4] # ok
dumb: list[int, str] = ['1', 2, '3', 4] ## not ok

# only one type of key and values
mapping = {'proxy': True, 'label': False, 'remote': True} # ok
dumb: = {1: '3', 'key': 'value', True: False} # not ok

# no "complex typing"!!! Use classes instead
config = {
    "app_name": "MyApp",
    "version": "1.0.0",
    "services": [
        {
            "name": "auth",
            "enabled": True,
            "endpoints": ["/login", "/register", "/logout"]
        },
        {
            "name": "data",
            "enabled": False,
            "endpoints": ["/fetch", "/upload"]
        },
        ...
    ]
}
```

Don't ever have “magic strings”, i.e., hardcoded strings in the code if they appear more than once. Use a strategy to collect them.

Ideally use typed code and Enums for strings

- › As you program reads in any string, convert it to an Enum immediately. Work with the enum in your code:

```
from enum import Enum

class UserRole(Enum):
    ADMIN = "admin"
    EDITOR = "editor"
    VIEWER = "viewer"

# Usage
role = UserRole.ADMIN
if role == UserRole.ADMIN:
    print("Access granted to admin features.")
```

If you need strings, e.g., pandas columns, use constants

- › If you cannot use enums, use constants, ideally separated in files or classes in some meaningful way.
- › For example, one file for each “table” in your data, each file contains the column names

```
from patterns.string_constants.schema1 import table1
from patterns.string_constants import schema1

def some_processing_fxd():
    df = pd.read_sql(f"""
        SELECT {table1.col2, table1.col3}
        FROM {schema1.name}.table1
        WHERE col1 > 10
        """, con=...)

    df['feature1'] = df[table1.col2] / (df[table1.col3] + df[table1.col2])

    return df['feature1']
```

Testing is crucial in data engineering projects as it ensures the accuracy, consistency, and reliability of data, which are vital for trustworthy insights and decision-making.

It also helps validate the scalability and robustness of data pipelines, preventing production issues and reducing risks early in the development process.

Test type	Description
A Unit Tests	• Purpose: Test individual components or functions in isolation. • Example: Testing a single function that transforms data, ensuring it handles edge cases like null or out-of-range inputs. • Tools: JUnit (Java), pytest (Python)
B Integration Tests	• Purpose: Validate interactions between components or modules. • Example: Testing the integration between a database and an API to ensure data is correctly fetched and processed. • Tools: Postman, REST-assured, pytest (with fixtures for dependencies).
C System Tests	• Purpose: Test the entire system end-to-end to verify it meets requirements. • Example: A test case that processes a batch of data through the entire ETL (Extract, Transform, Load) pipeline. • Tools: Selenium (for front-end testing), custom scripts for data pipelines
D Regression Tests	• Purpose: Ensure that new code changes do not break existing functionality. • Example: Running tests on a machine learning model pipeline after adding a new preprocessing step. • Tools: Automated test suites.
E Performance Tests	• Load Testing: Measure system performance under expected load. • Stress Testing: Evaluate how the system performs under extreme conditions. • Scalability Testing: Assess the system's ability to scale up or down.
F Acceptance Tests	• Purpose: Validate that the system meets business requirements. • User Acceptance Testing (UAT): Performed by end-users to ensure the system meets their needs. • Contract Acceptance Testing: Verify contractual obligations are met.
G A/B Testing	• Purpose: Compare two system versions to determine which performs better. • Example: Testing two recommendation algorithms to see which increases user engagement. • Tools: Experimentation platforms like Optimizely, custom scripts.

pytest is a versatile and powerful testing framework for Python, offering support for simple unit tests as well as complex functional testing with a focus on ease of use and scalability.

pytest has rich features and is ideal for unit testing

- › **Fixtures**: Simplifies test setup and teardown with reusable and configurable resources.
- › **Parameterization**: Enables running the same test with different data inputs efficiently.
- › **Plugin Ecosystem**: Offers extensive plugins for added functionality like test reporting, benchmarking, and parallel test execution.
- › **Assertions**: Provides rich and informative assertion introspection for debugging test failures.
- › **Test Discovery**: Automatically discovers tests in files and directories without requiring explicit configuration.
- › **Integration with CI/CD**: Works seamlessly with tools like GitHub Actions, Jenkins, and GitLab CI for automated testing.

Some example features

```
import pytest

def multiply(a, b):
    return a * b

@pytest.mark.parametrize("a, b, expected", [
    (2, 3, 6),
    (4, 5, 20),
    (1, 0, 0),
])
def test_multiply(a, b, expected):
    assert multiply(a, b) == expected

def divide(a, b):
    if b == 0:
        raise ValueError("Division by zero is not allowed")
    return a / b

def test_divide():
    assert divide(10, 2) == 5 # Pass
    assert divide(9, 3) == 3 # Pass
    with pytest.raises(ValueError, match="Division by zero is not allowed"):
        divide(5, 0)
```


When creating python projects, follow the typical structure to make it easier for others to work with.

Content overview

- › **Source Code** (`my_project/`): Contains the core Python code, organized into modules or packages.
- › **Tests** (`tests/`): Includes test cases for the codebase, often using a framework like pytest.
- › **Documentation** (`docs/`): Project guides, usage instructions, and API references.
- › **Configuration Files**: Includes `pyproject.toml`, `requirements.txt`, and setup files for dependencies and build settings.
- › **README.md**: A user-friendly overview of the project, with installation and usage instructions.
- › **License and Metadata Files**: License file for usage terms, `.gitignore` for ignored files, and CI/CD configurations if applicable.

Typical structure

```
my_project/
├── src/
│   └── my_project/
│       ├── __init__.py
│       ├── module1.py
│       ├── module2.py
│       └── utils.py
├── tests/
│   ├── test_module1.py
│   ├── test_module2.py
│   └── conftest.py
├── docs/
├── requirements.txt
├── pyproject.toml
├── setup.py
├── README.md
├── LICENSE
└── .gitignore
```

Source code directory
Project's Python package
Package initializer
Module 1
Module 2
Utility functions
Test cases
Tests for module1
Tests for module2
Shared test fixtures
Documentation files
Dependencies list
Build configuration and metadata
Legacy build configuration (optional)
Project overview and usage guide
License file
Git ignore rules

Dependency Injection

Dependency Injection (DI) is a design pattern in software development where an object's dependencies are supplied by an external entity rather than being hard-coded within the object itself. This approach allows for a decoupling of components, promoting more modular and flexible code. When an object relies on other objects to function, instead of creating those dependencies directly within the object, they are injected from outside, usually via constructors, setters, or through interface injection

1

Testability

This design pattern significantly enhances the testability of the code, as dependencies can be easily mocked or stubbed out during unit testing, allowing for more isolated and precise tests.

2

Maintainability

DI provides better maintainability, as the code becomes more modular, making it easier to update or replace components without affecting the entire system.

3

Re-useability

DI promotes reusability, as objects are no longer tightly coupled to their dependencies, allowing the same object to be reused in different contexts with different dependencies.

4

Extensibility

DI enhances extensibility, as it becomes simpler to extend the functionality of the system by injecting new dependencies or modifying existing ones without changing the codebase itself.

The main issue with dependency injection is the added complexity of managing and configuring dependencies, which can increase initial development overhead and make debugging more challenging if poorly implemented.

Tightly coupled dependencies cause problems

- › The **DataProcessor** class directly creates an instance of the **Database** class, hardcoding the dependency.
- › This tightly couples **DataProcessor** to **Database**, making it difficult to replace or mock the dependency during testing.
- › Hard to Mock Dependencies:
 - › You cannot easily test **DataProcessor** with mock data, as it always uses the real **Database** instance.
 - › For example, testing behavior with different data or failure scenarios (e.g., database connection issues) becomes cumbersome.
- › Rigid Structure: If you want **DataProcessor** to fetch data from another source (e.g., a REST API or a CSV file), you must modify the class to replace the hardcoded **Database** dependency.

A bad design example

```
class Database:
    def fetch_data(self):
        return ["item1", "item2", "item3"]
```

Dependency Inversion Principle (DIP):

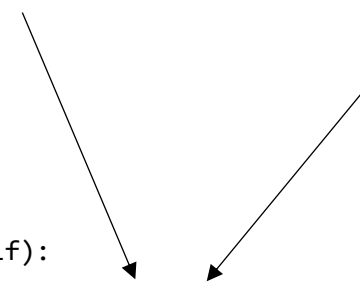
High-level modules do not depend on low-level modules; both depend on abstractions.

Single Responsibility Principle (SRP):

Classes focus on their primary responsibility, and dependencies are managed externally.

```
class DataProcessor:
    def __init__(self):
        ...
        self.database = Database()

    def process_data(self):
        data = self.database.fetch_data()
        return [item.upper() for item in data]
```



The refactored version follows dependency injection pattern by making the `data_source` a variable in the constructor of the other class, rather than being hardcoded.

The refactored version injects the db dependency

```
class DataProcessor:
    def __init__(self, data_source):
        self.data_source = data_source # Inject the dependency

    def process_data(self):
        data = self.data_source.fetch_data()
        return [item.upper() for item in data]

class Database:
    def fetch_data(self):
        return ["item1", "item2", "item3"]
```

This makes testing simple as you can mock inputs

```
import pytest
from my_project.data_processor import DataProcessor

class MockDataSource:
    def fetch_data(self):
        return ["mock1", "mock2", "mock3"]

def test_data_processor_with_mock_data():
    mock_data_source = MockDataSource()
    processor = DataProcessor(mock_data_source)
    result = processor.process_data()
    assert result == ["MOCK1", "MOCK2", "MOCK3"]

def test_data_processor_with_empty_data():
    class EmptyDataSource:
        def fetch_data(self):
            return []

    processor = DataProcessor(EmptyDataSource())
    result = processor.process_data()
    assert result == []
```

Premature optimization is the act of refining a system's performance before thoroughly understanding its actual bottlenecks or requirements.

This practice often results in wasted effort, increased code complexity, and misaligned priorities, which can hinder the overall success of a project. Effective optimization should be guided by profiling, real-world data, and a clear understanding of system behavior to ensure meaningful and impactful improvements.

- ▶ **Focus on Low-Impact Areas:** Premature optimization often involves spending significant time and effort refining parts of a system that contribute minimally to overall performance. This distracts from addressing higher-impact areas that could deliver substantial improvements. Identifying actual bottlenecks through profiling and observation is critical before any optimization work begins.
- ▶ **Increased Complexity:** Early optimization can lead to over-engineered solutions that are difficult to understand, maintain, or modify. This added complexity can result in more bugs, slower development cycles, and higher long-term costs, outweighing any performance gains.
- ▶ **Misaligned Priorities:** Overemphasizing optimization at an early stage can shift focus away from delivering core functionality, meeting deadlines, or addressing customer needs. It risks creating a product that is efficient in specific aspects but fails to meet overall project goals.
- ▶ **Lack of Real-World Context:** Premature optimization is often based on assumptions rather than real data, leading to solutions that solve theoretical problems instead of actual ones. Performance bottlenecks tend to emerge in unexpected areas, making early optimizations often irrelevant or wasteful.